AWS Resource Usage Tracker — Project Documentation

Overview

This project automates tracking of AWS resource usage (EC2, S3, Lambda, IAM) using a Bash script running on an EC2 instance, scheduled via cron for periodic, unattended reporting. This helps with visibility into active resources for cost management purposes.

Environment:

- Cloud Provider: AWS
- Region: Asia Pacific (Mumbai) — `ap-south-1`
- Compute: EC2 (Ubuntu)
- Automation: Cron

Step 1: EC2 Instance Setup

An EC2 instance was provisioned in the `ap-south-1` region to host and run the tracking script.

Instances observed in the account:

Name	Instance ID	State	Type	Status Check
Rtracker	i-0c66dc999a409614a	Running	t3.micro	3/3 checks passed
devops	i-035953b5cadd59b21	Running	t3.micro	3/3 checks passed
test	i-00865a9b04afed574	Running	t3.micro	3/3 checks passed

All instances passed status checks with no active alarms, and EC2 service health showed "Operating normally" for the region.

An IAM user (`faris`) was created with `AdministratorAccess` permissions to allow the AWS CLI to authenticate and query resources across services.

Step 2: Bash Script Creation

A shell script (`aws_resource_tracker.sh`) was written to query and report on AWS resources. The script covers:

- **S3 Buckets** — lists buckets and their sizes
- **EC2 Instances** — lists instance ID, state, and type
- **Lambda Functions** — lists deployed functions (none present at time of testing)
- **IAM Users** — lists IAM users on the account

Sample successful output:

```
==== S3 Buckets ====
2026-07-27 15:56:50 s3devopss3

==== EC2 Instances ====
+----------------------+---------+-------------+
|           DescribeInstances                  |
+----------------------+---------+-------------+
| i-0c66dc999a409614a  | running | t3.micro    |
| i-035953b5cadd59b21  | running | t3.micro    |
| i-00865a9b04afed574  | running | t3.micro    |
+----------------------+---------+-------------+

==== Lambda Functions ====

==== IAM Users ====
+----------------+
|   ListUsers    |
+----------------+
| faris          |
| shaihanath     |
+----------------+

==== Report Generated Successfully ====
```

Issues encountered and fixes during development

Issue	Cause	Fix
`--output: command not found`	Line break / quoting issue when editing the script manually	Corrected the `aws ... --output table` command onto a single properly escaped line
`aws_resource_tracker.sh: command not found`	Script wasn't in `$PATH` and wasn't called with a relative path	Ran with `./aws_resource_tracker.sh` after `chmod +x`
`InvalidAccessKeyId` / `InvalidClientTokenId` on all AWS calls	AWS CLI had no valid credentials configured	Created an IAM user (`faris`) with `AdministratorAccess`, generated an access key pair, and ran `aws configure`

Script permissions were set with:

```bash
chmod 777 aws_resource_tracker.sh   # later tightened to chmod +x
```

Step 3: Cron Job Setup

The script was scheduled to run automatically using `crontab`.

Steps performed:

```bash
crontab -e
```

Editor selected: `nano` (option 1).

Cron line added:

```
5 * * * * /home/ubuntu/aws_resource_tracker.sh >> /home/ubuntu/aws-tracker-logs/cron.log 2>&1
```

This schedules the script to run at minute 5 of every hour, redirecting both standard output and errors into a log file for later review.

Verified with:

```bash
crontab -l
```

Issue encountered: missing log directory

The first scheduled run produced no log file at all:

```
cat: /home/ubuntu/aws-tracker-logs/cron.log: No such file or directory
```

Root cause: cron's `>>` redirect needs the target folder to already exist — it can't create it on the fly, and the script's own `mkdir -p` line never got the chance to run because the redirect itself failed first.

Fix: The log directory was created manually ahead of time, and the log file path was simplified to a single file (`/home/ubuntu/aws_resource_tracker.logs`) for later runs.

Step 4: Verified Working Cron Job

Confirmed via `sudo journalctl -u cron` that cron successfully executed the script on schedule, repeatedly, at consistent intervals:

```
Jul 28 06:02:02 CRON[2734]: (ubuntu) CMD
(/home/ubuntu/aws_resource_tracker.sh >> /home/ubuntu/aws-tracker-
logs/cron.log 2>&1)
Jul 28 06:09:01 CRON[2856]: (ubuntu) CMD
(/home/ubuntu/aws_resource_tracker.sh >>
/home/ubuntu/aws_resource_tracker.logs/cron.log 2>&1)
Jul 28 06:10:01 CRON[2887]: (ubuntu) CMD
(/home/ubuntu/aws_resource_tracker.sh >>
/home/ubuntu/aws_resource_tracker.logs/cron.log 2>&1)
Jul 28 06:11:01 CRON[2917]: (ubuntu) CMD
(/home/ubuntu/aws_resource_tracker.sh >>
/home/ubuntu/aws_resource_tracker.logs 2>&1)
```

Checking the resulting log file confirmed multiple successful, automatic runs:

bash

```
cat /home/ubuntu/aws_resource_tracker.logs
```

```
==== S3 Buckets ====
2026-07-27 15:56:50 s3devopss3

==== EC2 Instances ====
| i-0c66dc999a409614a | running | t3.micro |
| i-035953b5cadd59b21 | running | t3.micro |
| i-00865a9b04afed574 | running | t3.micro |

==== Lambda Functions ====

==== IAM Users ====
| faris      |
| shaihanath |

==== Report Generated Successfully ====
```

Using `tail -f` on the log file showed the report regenerating automatically at each scheduled interval without manual intervention — confirming the cron job is running reliably and unattended.

Summary

Phase	Status
EC2 instance provisioned	✅ Complete
Bash script written and tested manually	✅ Complete
AWS CLI authentication configured	✅ Complete
Cron job scheduled	✅ Complete
Cron job verified running automatically	✅ Complete

The AWS Resource Usage Tracker is now fully automated: it runs on a defined schedule via cron, queries EC2, S3, Lambda, and IAM, and logs the results for ongoing resource and cost visibility.